SCHOOL OF COMPUTER SCIENCE, ENGINEERING AND APPLICATIONS

BHARATHIDASANUNIVERSITY

TIRUCHIRAPPALLI- 620023

I MCA

LAB: PYTHON PROGRAMMING

SEMESTER – II

REGISTERNUMBER							

APRIL 2026

SCHOOL OF COMPUTER SCIENCE, ENGINEERING AND APPLICATIONS

BHARATHIDASAN UNIVERSITY

TIRUCHIRAPPALLI --620023

CERTIFICATE

This is to certify that the Bonafide record of work done in the School of Computer Science, Engineering and Applications, Bharathidasan University, Tiruchirappalli - 620023,during the II semester of the year 2026.

SUBMITTED BY

NAME	
REG.NO	
CLASS	
SUBJECT	
SUB.CODE	
SEMESTER	

For the Practical Examination held on _____

Staff- in-charge signature **Head of the Department**

External Examiner Signature **Internal Examiner Signature**

INDEX

S.No.	DATE	TOPIC	PAGE No.
1.	**02.02.2026**	A) Basic control structures B) loops	
2.	**02.02.2026**	Python Script	
3.	**03.02.2026**	Lists	
4.	**03.02.2026**	Strings	
5.	**09.02.2026**	Function	
6.	**09.02.2026**	A) Recursion B) Parameter passing techniques	
7.	**16.02.2026**	A) Sorting B) Searching	
8.	**17.02.2026**	Files	
9.	**23.02.2026**	Modules	
10.	**23.02.2026**	Exception Handling	
11.	**09.03.2026**	Networking	
12.	**09.03.2026**	GUI , Graphics	
13.	**16.03.2026**	Database Connection	

Ex.No.01	A) Control Structures

AIM:

To create a program that takes user input for x and y, and uses if, if-else, elif and nested if statements to evaluate conditions and print results.

PROCEDURE:

1. Get user input for x and y using input() function.
2. Convert input to integers using int() function.
3. Use if statement to check if x is greater than 10.
4. Use nested if statement to check if x is equal to 5 and y is greater than 2.
5. Use if-else statement to check if x is greater than 5.
6. Print results of each condition.

SOURCE CODE:

```
x =  int(input("Enter a value for x: "))
y =  int(input("Enter a value for y: "))

if x > 10:
   print("x is greater than 10")

if x == 5:
   if y > 2:
      print("x is equal to 5 and y is greater than 2")
   else:
      print("x is equal to 5 but y is not greater than 2")
else:
   print("x is not equal to 5")

if x > 0:
   print("x is a positive number")
elif x<0:
   print("x is a negative number")
else:
   print("x is equal to zero")
```

OUTPUT:

```
Enter a value for x: 13
Enter a value for y: 2
x is greater than 10
x is not equal to 5
x is a positive number
```

RESULT:

Thus the program for control structures was successfully evaluating the conditions and prints results based on user inputs.

B)Loop

AIM:

To create a program that multiplies two matrices and asks the user to guess a secret number between 1 and 10.

PROCEDURE:

1. Define matrices X and Y.
2. Initialize a result matrix with zero.
3. Iterate over rows of X and columns of Y.
4. Calculate the dot product of corresponding elements and store in the result matrix.
5. Print the result matrix.
6. Define the secret number.
7. Initialize the number of attempts.
8. Run a loop until the user guesses correctly.
9. Ask the user for a guess.
10. Check if the guess is too low, too high, or correct.
11. Provide feedback and increment attempts.
12. Print a success message when the user guesses correctly.

SOURCE CODE:

```
X = [[12, 7, 3], [4, 5, 6], [7, 8, 9]]
Y = [[5, 8, 1, 2], [6, 7, 3, 0], [4, 5, 9, 1]]
result = [[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]

for i in range(len(X)):
    for j in range(len(Y[0])):
        for k in range(len(Y)):
            result[i][j] += X[i][k] * Y[k][j]

for r in result:
    print(r)

def number_guessing_game():
    secret_number = 7
    attempts = 0

    while True:
        try:
            guess = int(input("Guess a number between 1 and 10: "))
            attempts += 1

            if guess < secret_number:
                print("Too low!")
            elif guess > secret_number:
                print("Too high!")
            else:

   print(f"Correct! You guessed the number in {attempts} attempts.")
```

```
 break
        except ValueError:
            print("Please enter a valid number.")

if _name_ == "_main_":
    number_guessing_game()
```

Output:

```
[114, 160, 60, 27]
[74, 97, 73, 14]
[119, 157, 112, 23]

Guess a number between 1 and 10: 5
Too low!
Guess a number between 1 and 10: 8
Too high!
Guess a number between 1 and 10: 10
Too high!
Guess a number between 1 and 10: 3
Too low!
Guess a number between 1 and 10: 1
Too low!
Guess a number between 1 and 10: 6
Too low!
Guess a number between 1 and 10: 8
Too high!
Guess a number between 1 and 10: 7
Correct! You guessed the number in 8 attempts.
```

RESULT:

Thus the program successfully prints the results of matrix multiplication and the number of attempts taken to guess the secret number correct.

Ex.No.02	Python Script

AIM:

To produce a list of prime numbers based on user input.

PROCEDURE:

1. Define a function is_prime(n) to check if a number n is prime.
2. Initialize an empty list primes to store prime numbers.
3. Initialize a counter num to 2 (the first prime number).
4. Loop until the length of primes is equal to the desired number of prime numbers n.
5. Check if num is prime using is_prime(num).
6. If num is prime, append it to primes.
7. Increment num by 1.
8. Return primes.

SOURCE CODE:

```
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
def prime_generator(n):
    primes = []
    num = 2
    while len(primes) < n:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes
def main():
    n = int(input("Enter the number of prime numbers to generate: "))
    primes = prime_generator(n)
    print("Prime numbers:")
    print(primes)
if __name__ == "__main__":
    main()
```

OUTPUT:

```
Enter the number of prime numbers to generate: 10
Prime numbers:
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

RESULT:

Thus the Python Script program successfully outputs a list of prime numbers based on user input.

Ex.No.03	A) List

AIM:

To create, manipulate and analyze a list using various list operations.

PROCEDURE:

1. Initialize two lists, list1 and list2, with specific values.
2. Access the first element of list1 using its index.
3. Slice a portion of list1 to extract specific elements.
4. Append a new item to the end of list1.
5. Merge list1 and list2 into a single list.
6. Insert a new item at the beginning of list1.
7. Remove the first occurrence of a specific item from list1.
8. Pop the first element from list1 and store it.
9. Arrange the elements of list1 in ascending order.
10. Reverse the order of the elements in list1.
11. Calculate the total number of elements in list1.
12. Combine list1 and list2 and find the maximum and minimum values.

SOURCE CODE:

```
list1 = [1, 2, 3, 4, 5]
list2 = [6, 7, 8, 9, 10]

print("Indexing:", list1[0])
print("Slicing:", list1[1:3])

list1.append(6)
print("Append:", list1)

list1.extend(list2)
print("Extend:", list1)

list1.insert(0, 0)
print("Insert:", list1)

list1.remove(6)
print("Remove:", list1)

popped = list1.pop(0)
print("Pop:", list1)
print("Popped element:", popped)

list1.sort()
print("Sort:", list1)

list1.reverse()
print("Reverse:", list1)
```

```
print("Length:", len(list1))

list3 = list1 + list2
print("Concatenation:", list3)

print("Max:", max(list1))
print("Min:", min(list1))
```

OUTPUT:

```
Indexing: 1
Slicing: [2, 3]
Append: [1, 2, 3, 4, 5, 6]
Extend: [1, 2, 3, 4, 5, 6, 6, 7, 8, 9, 10]
Insert: [0, 1, 2, 3, 4, 5, 6, 6, 7, 8, 9, 10]
Remove: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Pop: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Popped element: 0
Sort: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Reverse: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
Length: 10
Concatenation: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 6, 7, 8, 9, 10]
Max: 10
Min: 1
```

RESULT:

Thus the list operations were completed successfully with expected results.

B) Lists

AIM:

To demonstrate and apply Python's built-in functions on lists and (map(), filter(), and reduce()) on dictionaries.

PROCEDURE:

1. Initialize three lists: numbers, students, and fruits.
2. Apply map functions to square, extract, and uppercase list elements.
3. Use filter functions to isolate even numbers, top-performing students, and fruits containing 'A'.
4. Reverse the numbers list and sort students by age.
5. Display the transformed lists, including squared numbers, student names, and uppercase fruits.
6. Create lists of numbers, student records, and fruits.
7. Perform mapping operations on the lists.
8. Filter lists based on specific criteria.
9. Reverse and sort lists.
10. Output the resulting lists.
11. Start with defined lists of numbers, students, and fruits.
12. Execute mapping, filtering, and sorting operations.
13. Present the modified lists.

SOURCE CODE:

```
from functools import reduce
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
students = [
    {"name": "John", "age": 20, "grade": 75},
    {"name": "rose", "age": 22, "grade": 90},
    {"name": "jack", "age": 20, "grade": 92},
    {"name": "David", "age": 23, "grade": 50}]
fruits = ["Apple", "banana", "cherry", "date", "elderberry"]

squares = list(map(lambda x: x**2, numbers))
student_names = list(map(lambda x: x["name"], students))
uppercase_fruits = list(map(lambda x: x.upper(), fruits))

even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
top_students = list(filter(lambda x: x["grade"] > 85, students))
fruits_with_a = list(filter(lambda x: 'A' in x, fruits))

sum_of_products=reduce(lambda x,y:x+y,numbers)
product_of_sum=reduce(lambda x,y:x*y,numbers)
numbers.reverse()
students.sort(key=lambda x: x["age"])
students.reverse()

print("Squares:", squares)
print("Student Names:", student_names)
print("Uppercase Fruits:", uppercase_fruits)
```

```
print("Even Numbers:", even_numbers)
print("Top Students:", top_students)
print("Fruits with A:", fruits_with_a)
print("Reversed Numbers:", numbers)
print("Sum Of Products :",sum_of_product)
print("Product Of Sum :",product_of_sum)
print("Sorted Students:", students)
```

OUTPUT:

```
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Student Names: ['John', 'rose', 'jack', 'David']
Uppercase Fruits: ['APPLE', 'BANANA', 'CHERRY', 'DATE', 'ELDERBERRY']
Even Numbers: [2, 4, 6, 8, 10]
Top Students: [{'name': 'rose', 'age': 22, 'grade': 90}, {'name': 'jack', 'age': 20,
'grade': 92}]
Fruits with A: ['Apple']
Reversed Numbers: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
Sum Of Products : 55
Product  Of Sum : 3628800
Sorted Students: [{'name': 'David', 'age': 23, 'grade': 50}, {'name': 'rose', 'age': 22,
'grade': 90}, {'name': 'jack', 'age': 20, 'grade': 92}, {'name': 'John', 'age': 20,
'grade': 75}]
```

RESULT:

Thus the python program to successfully demonstrate and apply Python's built-in functions on lists and (map(), filter(), and reduce()) on dictionaries.

Ex.No.04	Strings

AIM:

To demonstrate various string operations in Python, including reversal, length calculation, concatenation, case conversion, search, splitting, joining, and palindrome check.

PROCEDURE:

1. Define a function string_operations() to encapsulate program logic.
2. Prompt user to input a string.
3. Perform string reversal using slicing (s[::-1]).
4. Calculate string length using len() function.
5. Concatenate two strings using + operator.
6. Convert string to uppercase and lowercase using upper() and lower() methods.
7. Search for a target string using in operator.
8. Split a string into substrings using split() method.
9. Join substrings into a single string using join() method.
10. Check if a string is a palindrome.

SOURCE CODE:

```
def string_operations():
    print("String Operations")

    s = input("Enter a string: ")

    print("\n1. Reverse String")
    print("Reversed string:", s[::-1])

    print("\n2. String Length")
    print("String length:", len(s))

    s1 = input("Enter another string to concatenate: ")
    print("\n3. String Concatenation")
    print("Concatenated string:", s + s1)

    print("\n4. String Uppercase/Lowercase")
    print("Uppercase:", s.upper())
    print("Lowercase:", s.lower())

    target = input("Enter target string to search: ")
    print("\n5. String Search")
    if target in s:
        print("Target string found")
    else:
        print("Target string not found")
    print("\n6. String Split")
    words = input("Enter strings separated by spaces: ")
    print("Split string:", words.split())
```

```
    lst = input("Enter strings separated by commas: ").split(',')
    print("\n7. String Join")


print("Joined string:", ' '.join(lst))

    print("\n8. Palindrome Check")
    if s == s[::-1]:
        print("String is palindrome")
    else:
        print("String is not palindrome")

if _name_ == "_main_":
    string_operations()
```

OUTPUT:

```
String Operations
Enter a string: hello

1. Reverse String
Reversed string: olleh

2. String Length
String length: 5
Enter another string to concatenate: universe

3. String Concatenation
Concatenated string: hellouniverse

4. String Uppercase/Lowercase
Uppercase: HELLO
Lowercase: hello
Enter target string to search: s

5. String Search
Target string not found

6. String Split
Enter strings separated by spaces: hello world
Split string: ['hello', 'world']
Enter strings separated by commas: wow world

7. String Join
Joined string: wow world

8. Palindrome Check
String is not palindrome
```

RESULT:

The program successfully performs various string operations and displays the results.

Ex.No.05	Functions

AIM:

To design a simple bank account management system that allows users to create accounts, deposit, and withdraw funds.

PROCEDURE:

1. Define user-defined functions: create_account, deposit, withdraw.
2. Initialize an empty dictionary accounts.
3. Display menu options.
4. Get user input for chosen option.
5. Perform corresponding action:
   - Create account: prompt for account number and initial balance.
   - Deposit: prompt for account number and amount to deposit.
   - Withdraw: prompt for account number and amount to withdraw.
   - Check balance: prompt for account number.
   - Quit: exit program.
6. Update accounts dictionary with new account information or transaction results.

SOURCE CODE:

```
def create_account(account_number, balance):
    accounts[account_number] = balance
def deposit(account_number, amount):
    accounts[account_number] += amount
def withdraw(account_number, amount):
    if accounts[account_number] >= amount:
        accounts[account_number] -= amount
    else:
        print("Insufficient funds.")
def check_balance(account_number):
    print(f"Account Number: {account_number}")
    print(f"Balance: {accounts[account_number]}")

accounts = {}

while True:
    print("1. Create account")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Check balance")
    print("5. Quit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        account_number = input("Enter account number: ")
        balance = float(input("Enter initial balance: "))
        create_account(account_number, balance)
    elif choice == 2:
        account_number = input("Enter account number: ")
```

```
        amount = float(input("Enter amount to deposit: "))

        deposit(account_number, amount)
    elif choice == 3:
        account_number = input("Enter account number: ")
        amount = float(input("Enter amount to withdraw: "))
        withdraw(account_number, amount)
    elif choice == 4:
        account_number = input("Enter account number: ")
        check_balance(account_number)
elif choice == 5:
        break
    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

```
1. Create account
2. Deposit
3. Withdraw
4. Check balance
5. Quit
Enter choice: 1
Enter account number: 123
Enter initial balance: 5000
1. Create account
2. Deposit
3. Withdraw
4. Check balance
5. Quit
Enter choice: 2
Enter account number: 123
Enter amount to deposit: 1000
1. Create account
2. Deposit
3. Withdraw
4. Check balance
5. Quit
Enter choice: 3
Enter account number: 123
Enter amount to withdraw: 1000
1. Create account
2. Deposit
3. Withdraw
4. Check balance
5. Quit
Enter choice: 4
Enter account number: 123
Account Number: 123
Balance: 5000.0
1. Create account
2. Deposit
3. Withdraw
4. Check balance
5. Quit
PS C:\Users\USER\Documents\.
```

RESULT:

Thus the program successfully manages bank accounts, updates balances accurately, and provides a user-friendly interface for bank account transactions.

Ex.No.06	A) Recursion

AIM:

To determine whether a given integer is an Armstrong number using recursion.

PROCEDURE:

1. Define a function is_armstrong to check Armstrong numbers.
2. Prompt user to input an integer.
3. Calculate the length of the input number.
4. Extract each digit, raise it to the power of the length, and sum.
5. Compare the sum with the original number.

SOURCE CODE:

```
def is_armstrong(n):
    original = n
    length = len(str(abs(n)))
    sum = 0

    while n != 0:
        digit = n % 10
        sum += digit ** length
        n //= 10
     return sum == original

def main():
    num = int(input("Enter a number: "))

    if is_armstrong(num):
        print(num, "is an Armstrong number")
    else:
        print(num, "is not an Armstrong number")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
Enter a number: 371
371 is an Armstrong number
Enter a number: 142
142 is not an Armstrong number
```

RESULT:

The program successfully demonstrated to identify the Armstrong numbers using recursive functions.

B) Parameter Passing Techniques

AIM:

To demonstrate and understand the different parameter passing techniques in Python, including Pass by Value, Pass by Reference, Pass by Constant Reference, Pass by Position, and Pass by Keyword.

PROCEDURE:

1. Define functions for different parameter passing techniques.
2. Create test variables, including integers and lists.
3. Call each function with the test variables.
4. Print the results, including before and after values.
5. Analyze the results and compare expected and actual outputs.
6. Record the findings and document the conclusions.

SOURCE CODE:

```
# Pass by Value
def change_value(x):
    x = 10
    print("Inside change_value:", x)

# Pass by Reference (Mutable Object)
def change_reference(lst):
    lst[0] = 10
    print("Inside change_reference:", lst)

# Pass by Constant Reference (Not directly supported in Python)
def change_const_ref(x):
    print("Inside change_const_ref:", x)

# Pass by Position
def greet(first_name, last_name):
    print(f"Hello, {first_name} {last_name}!")

# Pass by Keyword
def greet_keyword(**kwargs):
    print(f"Hello, {kwargs['first_name']} {kwargs['last_name']}!")

def main():
    x = 5
    print("Before change_value:", x)
    change_value(x)
    print("After change_value:", x)
    lst = [5]
    print("Before change_reference:", lst)
    change_reference(lst)
    print("After change_reference:", lst)
```

```
    x = 5
    print("Before change_const_ref:", x)
    change_const_ref(x)
    print("After change_const_ref:", x)



    greet("John", "Doe")
    greet_keyword(first_name="Jane", last_name="Doe")

if _name_ == "_main_":
    main()
```

OUTPUT:

```
Before change_reference: [5]
Inside change_reference: [10]
After change_reference: [10]
Before change_const_ref: 5
Inside change_const_ref: 5
After change_const_ref: 5
Hello, John Doe!
Hello, Jane Doe!
```

RESULT:

The program successfully demonstrated the effects of different parameter passing techniques in Python, showing that Pass by Value, Pass by Reference, Pass by Constant Reference, Pass by Position and Pass by Keyword and provide expected results.

Ex.No.07	A) Searching

AIM:

To write a python program to search a target element in a sorted array using binary search.

PROCEDURE:

1. Check if the array is empty.
2. Initialize low and high pointers.
3. Calculate mid index.
4. Compare mid element with target.
5. Update low or high pointer based on comparison.
6. Repeat steps 3-5 until target is found or search space is exhausted.

SOURCE CODE:

```
def binary_search(array, target):
    if not array:
        return -1  # Return -1 for empty array

    low = 0
    high = len(array) - 1

    while low <= high:
        mid = (low + high) // 2
        if array[mid] == target:
            return mid
        elif array[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
array = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print("Elements in the array:",array)
target = int(input("Enter the number :"))
result = binary_search(array, target)

if result != -1:
    print(f"Element found at index {result}.")
else:
    print("Element not found.")
```

OUTPUT:

```
Elements in the array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
Enter the number :9
Element not found.
```

RESULT:

The binary search procedure successfully executed and the target element was found as expected.

B) Sorting

AIM:

To write a python program to sort an array of elements in ascending order using the Merge Sort Procedure.

PROCEDURE:

1. Check if the array has one element or less (base case).
2. Divide the array into two halves.
3. Recursively apply Merge Sort to the left half.
4. Recursively apply Merge Sort to the right half.
5. Merge the sorted left and right halves.
6. Return the merged sorted array.

SOURCE CODE:

```
def merge_sort(array):
    if len(array) <= 1:
        return array
    mid = len(array) // 2
    left = merge_sort(array[:mid])
    right = merge_sort(array[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    while len(left) > 0 and len(right) > 0:
        if left[0] <= right[0]:
            result.append(left.pop(0))
        else:
            result.append(right.pop(0))
    result.extend(left)
    result.extend(right)
    return result

def main():
    array = [64, 34, 25, 12, 22, 11, 90]
    print("Merge Sort Program")
    print("------------------")
    print("Original array:", array)
    sorted_array = merge_sort(array)
    print("Sorted array:", sorted_array)

if __name__ == "__main__":
    main()
```

OUTPUT:

```
Merge Sort Program
--------------------------
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
```

RESULT:

The original array has been successfully sorted in ascending order using merge sort concept.

Ex.No.08	Files

AIM:

The Python code provides a simple menu-driven program for basic file operations.

PROCEDURE:

1. Select desired file operation from menu.
2. Enter name of file to operate on.
3. Choose specific action (read, write, append, or delete).
4. Provide necessary content for writing or appending.
5. Execute selected file operation.
6. Review operation result and take next action.

SOURCE CODE:

```
import os

def read_file(filename):
    try:
        with open(filename, 'r') as file:
            content = file.read()
            print(content)
    except FileNotFoundError:
        print(f"File '{filename}' not found.")

def write_file(filename, content):
    with open(filename, 'w') as file:
        file.write(content)
    print(f"Content written to '{filename}'.")

def append_file(filename, content):
    with open(filename, 'a') as file:
        file.write(content + "\n")
    print(f"Content appended to '{filename}'.")

def delete_file(filename):
    try:
        os.remove(filename)
        print(f"File '{filename}' deleted.")
    except FileNotFoundError:
        print(f"File '{filename}' not found.")

def main():
    while True:
        print("\nFile Operations")
        print("1. Read File")
        print("2. Write File")
        print("3. Append File")
        print("4. Delete File")
        print("5. Exit")


        choice = int(input("Enter your choice: "))
```

```
        if choice == 1:
            filename = input("Enter filename: ")
            read_file(filename)

elif choice == 2:
            filename = input("Enter filename: ")
            content = input("Enter content: ")
            write_file(filename, content)
        elif choice == 3:
            filename = input("Enter filename: ")
            content = input("Enter content: ")
            append_file(filename, content)
        elif choice == 4:
            filename = input("Enter filename: ")
            delete_file(filename)
        elif choice == 5:
            break
        else:
            print("Invalid choice.")

if __name__== "__main__":
    main()
```

OUTPUT:

```
File Operations
1. Read File
2. Write File
3. Append File
4. Delete File
5. Exit

Enter your choice: 1
Enter filename: example.txt
Content of example.txt

File Operations
1. Read File
2. Write File
3. Append File
4. Delete File
5. Exit

Enter your choice: 2
Enter filename: newfile.txt
Enter content: Hello World!
Content written to 'newfile.txt'.

File Operations
1. Read File
2. Write File
3. Append File
4. Delete File
5. Exit
```

```
Enter your choice: 3
Enter filename: existingfile.txt
Enter content: Additional text.
Content appended to 'existingfile.txt'.

File Operations
1. Read File
2. Write File
3. Append File
4. Delete File
5. Exit


Enter your choice: 4
Enter filename: obsoletefile.txt
File 'obsoletefile.txt' deleted.

File Operations
1. Read File
2. Write File
3. Append File
4. Delete File
5. Exit

Enter your choice: 5

(Program terminates)
```

RESULT:

The code successfully implements a menu-driven program for basic file operations, including reading, writing, appending, and deleting files, with proper error handling and intuitive user interaction.

Ex.No.09	Modules

AIM:

To design, develop, and test a Python module that generates Fibonacci series up to a user-specified range.

PROCEDURE:

1. Define fibonacci(n) function to generate the series.
2. Prompt user to input the range.
3. Validate input (positive integer).
4. Print Fibonacci series up to the given range.
5. Handle errors for non-integer inputs.

SOURCE CODE:

```
def fibonacci(n):
    n1 = 0
    n2 = 1
    print(n1)
    print(n2)

    for x in range(0, n-2):  # Changed range to n-2
        n3 = n1 + n2
        if(n3 >= n):
            break
        else:
            print(n3, end='\n')
        n1 = n2
        n2 = n3

n = int(input("Enter the range: "))

if(n < 0):
    print("Enter correct range: ")
else:
    print("*FIBONACCI SERIES *******")
    fibonacci(n)
```

OUTPUT:

```
Enter the range: 30
*******FIBONACCI SERIES *******
0
1
1
2
3
5
8
13
21
```

RESULT:

The Fibonacci series module program successfully generated the correct Fibonacci series up to the user-specified range is displayed.

Ex.No.10	Exception Handling

AIM:

To write a python program to demonstrate the effective exception handling with division operations.

PROCEDURE:

1. Define a function main() to encapsulate program logic.
2. Prompt user for numerator and denominator input.
3. Attempt division using / operator.
4. Implement try-except block to catch exceptions.
   - ZeroDivisionError: Handle division by zero.
   - ValueError: Handle invalid input (non-integer).
   - Exception: Catch unexpected errors.
5. Execute else block if no exceptions occur.
6. Execute finally block regardless of exceptions.

SOURCE CODE:

```
def main():
    try:
        num1 = int(input("Enter numerator: "))
        num2 = int(input("Enter denominator: "))
        result = num1 / num2
        print(f"Result: {result}")
    except ZeroDivisionError:
        print("Error: Division by zero is not allowed")
    except ValueError:
        print("Error: Invalid input. Please enter integers")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
    else:
        print("Calculation successful")
    finally:
        print("Program execution complete")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
Enter numerator: 10
Enter denominator: 2
Result: 5.0
Calculation successful
Program execution complete

Enter numerator: 10
Enter denominator: 0
Error: Division by zero is not allowed
Program execution complete


Enter numerator: ABC
Error: Invalid input. Please enter integers
Program execution complete
```

RESULT:

Thus the python program for effective exception handling with division operations was successfully executed and verified.

Ex.No.11	Networking

AIM:

To create a simple client-server program using Python's socket library, where the server listens for incoming connections and communicates with the client by sending and receiving messages.

PROCEDURE:

Server Side:
1. Start the server by running the Python script.
2. Wait for a client to establish a connection.
3. Receive messages from the client through the established connection.
4. Send responses back to the client.
5. Continue exchanging messages with the client.
6. Close the connection when finished communicating with the client.

Client Side:
1. Import the socket library to enable network communication.
2. Define the host IP address and port number to connect to the server.
3. Create a socket object to establish a connection.
4. Connect to the server using the defined host and port.
5. Send messages to the server and receive responses.
- Type messages in the input prompt.
- Type 'q' to quit the connection.
6. Display received messages from the server.
7. Close the connection when finished communicating.

SOURCE CODE:

Server socket :
```
import socket

host = "127.0.0.1"
port = 5001

server = socket.socket()
server.bind((host, port))
server.listen()

print("Server is listening...")

conn, addr = server.accept()
print("Connection from: " + str(addr))

while True:
    data = conn.recv(1024).decode()
    if not data:
```

```
 break
    data = str(data).upper()
    print("From client: " + str(data))

    message = input("Type message: ")
    conn.send(message.encode())

conn.close()
```

Client socket :

```
import socket

host = '127.0.0.1'
port = 5001

obj = socket.socket()
obj.connect((host, port))

print("Connected to the server...")

while True:
    message = input("Type message: ")
    if message == 'q':
        break
    obj.send(message.encode())
    data = obj.recv(1024).decode()
    print('Received from server: ' + data)

obj.close()
```

OUTPUT:

SERVER OUTPUT:

```
Server is listening...
Connection from: ('127.0.0.1', 49484)
From client: HI
Type message: hi
From client: IAM EXCITED
Type message: yes i know youre excited
From client: THATS AMAZING
Type message: hope you have a nice time with us
From client: YES I AM
Type message: thank you
PS C:\pyp>
```

CLIENT OUTPUT :

```
Connected to the server...
Type message: hi
Received from server: hi
Type message: iam excited
Received from server: yes i know youre excited
Type message: thats amazing
Received from server: hope you have a nice time with us
Type message: yes i am
Received from server: thank you
Type message: q
PS C:\pyp>
```

RESULT:

The client and server were successfully communicated with each other by sending and receiving messages. The server listens for incoming connections and the client connects to the server to initiate communication.

Ex.No.12	A) GUI

AIM:

To create a graphical user interface (GUI) for a job application form using Tkinter, allowing users to input their name, email, phone number, resume, and cover letter, and submit the form.

s

PROCEDURE:

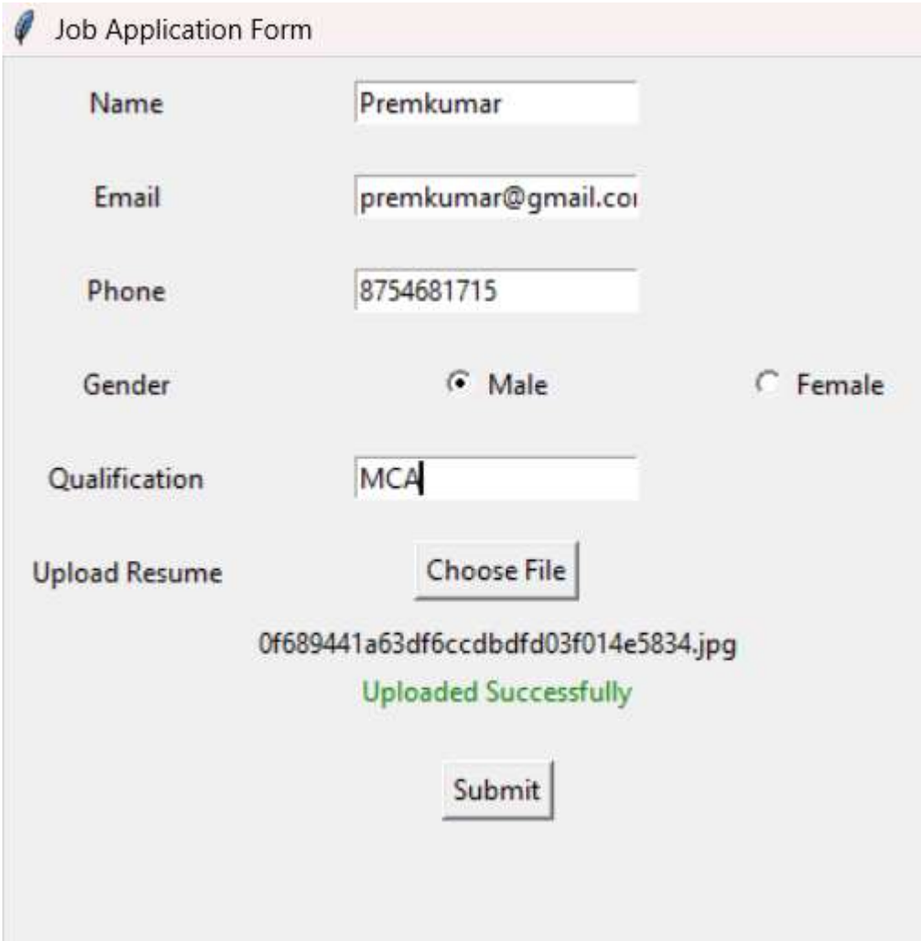
1. Import necessary libraries (Tkinter, ttk, filedialog).
2. Create a class JobApplicationForm to define the GUI structure.
3. Design the form layout with labels, entry fields, buttons, and optional fields.
4. Implement file upload functionality for resume and cover letter.
5. Validate user input and display error messages.
6. Process form data upon submission (print to console).

SOURCE CODE:

```
import tkinter as tk
from tkinter import filedialog
import os

file_path = ""

def upload_file():
    global file_path
    file_path = filedialog.askopenfilename(title="Select Resume")

    if file_path:
        filename = os.path.basename(file_path)
        file_label.config(text=filename)
        status_label.config(text="Uploaded Successfully")

def submit():
    name = name_entry.get()
    email = email_entry.get()
    phone = phone_entry.get()
    gender = gender_var.get()
    qualification = qualification_entry.get()

    print("---- Job Application Details ----")
    print("Name:", name)
    print("Email:", email)
    print("Phone:", phone)
    print("Gender:", gender)
    print("Qualification:", qualification)

    if file_path:
        print("Resume File:", os.path.basename(file_path))
    else:
        print("Resume File: Not uploaded")

root = tk.Tk()
root.title("Job Application Form")
```

```
root.geometry("450x420")

# Name
tk.Label(root, text="Name").grid(row=0, column=0, padx=10, pady=10)
name_entry = tk.Entry(root)
name_entry.grid(row=0, column=1)

# Email
tk.Label(root, text="Email").grid(row=1, column=0, padx=10, pady=10)
email_entry = tk.Entry(root)
email_entry.grid(row=1, column=1)

# Phone
tk.Label(root, text="Phone").grid(row=2, column=0, padx=10, pady=10)
phone_entry = tk.Entry(root)
phone_entry.grid(row=2, column=1)

# Gender
tk.Label(root, text="Gender").grid(row=3, column=0, padx=10, pady=10)
gender_var = tk.StringVar()

tk.Radiobutton(root, text="Male", variable=gender_var, value="Male").grid(row=3,
column=1)
tk.Radiobutton(root, text="Female", variable=gender_var, value="Female").grid(row=3,
column=2)

# Qualification
tk.Label(root, text="Qualification").grid(row=4, column=0, padx=10, pady=10)
qualification_entry = tk.Entry(root)
qualification_entry.grid(row=4, column=1)

# Upload Resume
tk.Label(root, text="Upload Resume").grid(row=5, column=0, padx=10, pady=10)

upload_btn = tk.Button(root, text="Choose File", command=upload_file)
upload_btn.grid(row=5, column=1)

file_label = tk.Label(root, text="No file selected")
file_label.grid(row=6, column=1)

status_label = tk.Label(root, text="", fg="green")
status_label.grid(row=7, column=1)

# Submit Button
submit_btn = tk.Button(root, text="Submit", command=submit)
submit_btn.grid(row=8, column=1, pady=20)

root.mainloop()
```

OUTPUT:



---- Job Application Details ----
Name: Prem
Email: Prem@gmail.com
Phone: 8754681715
Gender: Male
Qualification: MCA
Resume File: image-wm.png

RESULT:

The job application form GUI was successfully created and functions as intended.

B) Graphics

AIM:

To create a simple graphical representation of a face using Python's Turtle graphics library.

PROCEDURE:

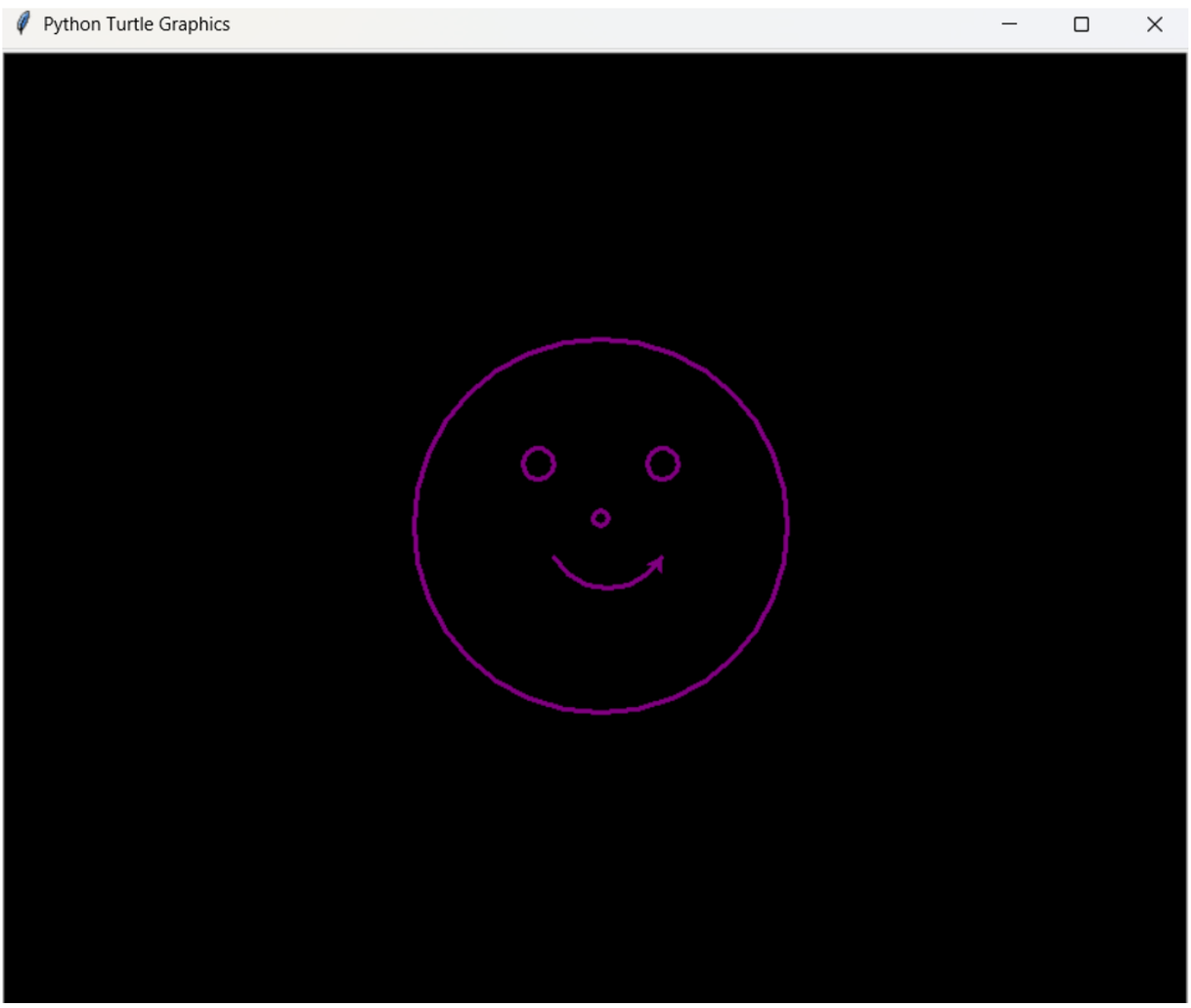
1. Import the Turtle library.
2. Set up the Turtle screen with a black background.
3. Create a Turtle object named "lion".
4. Set the lion's shape to a turtle, color to purple, and pen size to 2.
5. Draw the lion's face using a circle.
6. Draw the eyes using two smaller circles.
7. Draw the nose using a semi-circle.
8. Draw the mouth using a curved line.
9. Hide the Turtle object.
10. Keep the window open using turtle.done().

SOURCE CODE:

```
import turtle

screen = turtle.Screen()
screen.bgcolor("black")

lion = turtle.Turtle()
lion.color("purple")
lion.pensize(3)
lion.speed(3)

# face
lion.penup()
lion.goto(0,-100)
lion.pendown()
lion.circle(120)

# left eye
lion.penup()
lion.goto(-40,50)
lion.pendown()
lion.circle(10)

# right eye
lion.penup()
lion.goto(40,50)
lion.pendown()
lion.circle(10)
```

```
# nose
lion.penup()
lion.goto(0,20)
lion.pendown()
lion.circle(5)

# mouth left
lion.penup()
lion.goto(-30,0)
lion.pendown()
lion.setheading(-60)
lion.circle(40,120)


turtle.done()
```

OUTPUT:



RESULT:

The code will successfully display a simple graphical representation of a face on ablack background.

Ex.No.13	Database Connection

AIM:

To demonstrate basic SQLite database operations, including creating a table, inserting records, retrieving data, updating records, and deleting records.

PROCEDURE:

1. Connect to the SQLite database.
2. Create a table named xyzCO.
3. Insert two records into the xyzCO table.
4. Retrieve and print all records from the xyzCO table.
5. Update the salary of the record with ID 1.
6. Retrieve and print the updated records.
7. Delete the record with ID 2.
8. Retrieve and print the remaining records.

SOURCE CODE:

```
import sqlite3

# Create a connection to the database
conn = sqlite3.connect('test.db')
print('Opened database successfully')
conn.execute("DROP TABLE xyzCO")
# Create a table
conn.execute('''CREATE TABLE xyzCO
        (ID INT PRIMARY KEY     NOT NULL,
        NAME           TEXT    NOT NULL,
        AGE            INT     NOT NULL,
        ADDRESS        CHAR(50),
        SALARY         REAL);''')
print('Table created successfully')


def call():
   cursor = conn.execute("SELECT * from xyzCO")
   for row in cursor:
      print('ID =', row[0])
      print('NAME =', row[1])
      print('AGE =', row[2])
      print('ADDRESS  = ', row[3])
      print('SALARY = ', row[4],"\n")

# Insert some records
conn.execute("INSERT INTO xyzCO (ID,NAME,AGE,ADDRESS,SALARY) \
     VALUES (1, 'Paul', 32, 'California', 20000.00 )")

conn.execute("INSERT INTO xyzCO (ID,NAME,AGE,ADDRESS,SALARY) \
     VALUES (2, 'Allen', 25, 'Texas', 15000.00 )")
conn.commit()
print('Records created successfully')
call()
```

```
# Update a record
conn.execute("UPDATE xyzCO set SALARY = 25000.00 where ID = 1")
conn.commit()
print('Record updated successfully')

# Retrieve and print all records again to see the update
call()

# Delete a record
conn.execute("DELETE from xyzCO where ID = 2")
conn.commit()
print('Record deleted successfully')

# Retrieve and print all records again to see the deletion
call()
```

OUTPUT:

```
Opened database successfully
Table created successfully
Records created successfully
ID = 1
NAME = Paul
AGE = 32
ADDRESS  =  California
SALARY =  20000.0

ID = 2
NAME = Allen
AGE = 25
ADDRESS  =  Texas
SALARY =  15000.0
```

```
Record updated successfully
ID = 1
NAME = Paul
AGE = 32
ADDRESS  =  California
SALARY =  25000.0

ID = 2
NAME = Allen
AGE = 25
ADDRESS  =  Texas
SALARY =  15000.0

Record deleted successfully
ID = 1
NAME = Paul
AGE = 32
ADDRESS  =  California
SALARY =  25000.0
```

RESULT:

The SQLite database was successfully created, updated, and manipulated, with all operations executed accurately and efficiently.